

# **Autonomous Agile Aerobatics on Micro Quadcopters**

Deep Reinforcement Learning and Onboard Embedded Edge Inference

**Andreas Panagopoulos**

Department of Electrical and Computer Engineering, University of Patras, Greece

`andreas.panagopoulos@ac.upatras.gr`

Technical Project Presentation – 5th AI-Hub Competition (2026)

Open-Source Repository: [github.com/andrupe/Quadcopter-flip](https://github.com/andrupe/Quadcopter-flip)

# Motivation: An Extreme Control & Engineering Challenge

## Why Agile Micro Quadcopters?

- Palm-sized micro quadcopters are mechanically compliant and safe around humans, ideal for indoor confined spaces.
- Minimal rotational inertia enables massive angular acceleration ( $> 1,100^\circ/\text{s}$ ).
- **Underactuated Dynamics:** Four fixed propellers can only push upward relative to the body. To translate or stop, the entire vehicle must tilt.
- **The Core Value:** Performing a  $360^\circ$  power flip is an extreme control benchmark, testing the absolute limits of autonomy.

## The Primary Engineering Hurdle

- **Maneuver Complexity:** High-speed aggressive maneuvers push quadcopter flight dynamics far beyond linear regimes, involving violent rotational rates, severe non-linear aerodynamic drag, and minimal thrust control authority during inverted flight.
- **Project Goal:** Execute the complete pipeline, sensing, Kalman state filtering, and neural policy inference, **100% onboard** a single low-power microcontroller.
- **Thrust Headroom Reality:** Baseline 16 mm motors have marginal thrust ( $T/W \approx 1.85$ ), making aerobatics very hard. Upgraded motors raise mass beyond 33 g (to  $\sim 41$  g) but provide essential control authority ( $T/W \approx 2.97$ ).

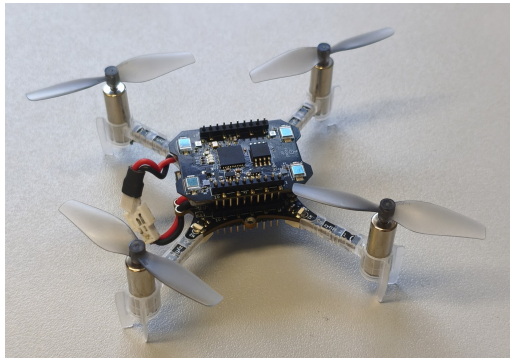
# Experimental Setup: The Bitcraze Crazyflie 2.1 Platform

## Palm-Sized Micro Quadcopter

- **Dimensions & Mass:** 9 cm rotor-to-rotor span; all-up mass  $\sim 27 - 33$  g in stock configuration.
- **Propulsion:** 4 brushed coreless DC motors (16 mm baseline) with direct-drive micro propellers.
- **Power:** 1-cell (1S) LiPo battery (nominal 3.7 V).
- **Sensors:** BMI088 IMU (1 kHz rates) + Lighthouse deck.

## Constrained Embedded Brain

- **STM32F405 MCU:** ARM Cortex-M4 @ 168 MHz, 192 KB RAM, 1024 KB Flash.
- **No Linux / No Companion Coprocessor:** 100% onboard execution without offboard assistance.
- **Real-Time OS:** FreeRTOS schedules radio, EKF, PWM, and neural network inference concurrently.



Crazyflie 2.1 with top-mounted Lighthouse v2 deck

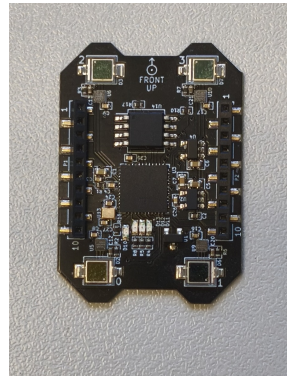
# Experimental Setup: Lighthouse Tracking & The Inversion Dilemma

## How Lighthouse Optical Tracking Works

- **External Base Stations:** Two Lighthouse v2 stations sweep rotating infrared laser beams across the flying volume.
- **Top-Deck Optical Receiver:** 4 photodiodes on the top deck capture pulse arrival times to solve 3D  $(x, y, z)$  position onboard via Extended Kalman Filter (EKF).

## The Inversion Tracking Blackout

- **Top-Deck Mount:** Photodiodes face up to track ceiling and wall base stations during normal upright flight.
- **Acrobatic Vulnerability:** During a  $360^\circ$  flip, the drone rotates past  $90^\circ$  tilt—the photodiodes face the floor and completely lose line of sight!
- **The Failure Mode:** Without optical fixes, standard EKF estimates diverge, causing standard controllers to cut throttle and crash.



Lighthouse deck with 4 corner photodiodes (0–3)

# Project Timeline: From Simulation Struggles to Hardware

## Chronological Development Phases

- **Phase 1: Dynamics Modeling & Singularity Fixes**  
Rigid-body kinematics; tackling CPU throughput limits and Euler angle gimbal lock during inverted flight.
- **Phase 2: JAX Parallelization & Reference Generator**  
Rewriting simulation in JAX for parallel rollouts; building an on-the-fly geometric reference generator.
- **Phase 3: Controller Architecture Evolution**  
Iterating from pure PPO to decoupled PPO-PID and recurrent latent history encoding.
- **Phase 4: Sim-to-Real Transfer & Flight Diagnostics**  
Lab flight cage tests, analyzing radio telemetry logs, implementing firmware crash safeguards, and achieving reliable flips.

## Early Simulation Roadblocks & Solutions

- **Why Not Euler Angles?** Standard roll, pitch, yaw  $(\phi, \theta, \psi)$  suffer mathematical gimbal lock at  $\pm 90^\circ$  and discontinuous  $180^\circ$  angle jumps during inversion. All dynamics and rewards were rebuilt using continuous **unit quaternions**  $q \in \mathbb{S}^3$ , handling double-cover ambiguities.
- **From Reward Shaping to Geometric Trajectories:**  
Initially attempted pure reward shaping to coax a flip; but finding a flip from sparse rewards is extremely brittle. Transitioned to an analytic geometric controller generating smooth reference curves on the fly across multiple families (hover, waypoints, Lissajous, flips).

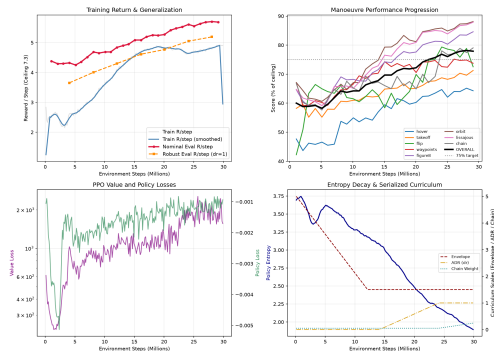
# Simulation Parallelization: JAX Rewrite & Training Dynamics

## Parallelizing Training with JAX

- Reinforcement learning requires millions of trial-and-error interactions to discover robust control policies.
- Rewrote the simulation environment in **JAX** to step multiple vectorized environments simultaneously.
- Even on CPU, vectorization provided a  $\sim 3\times$  to  $5\times$  **speedup** over serial stepping, enabling fast hyperparameter tuning.

## Training Dynamics: Compute-Bound Progress

- As seen on the right, episode rewards climb steadily from  $\sim 160$  to  $\sim 1,500$  across 30 million steps.
- The curve exhibits an **upward trend without plateauing**, indicating that training is **compute-bound**: additional training time would yield further performance gains.
- Retained tuned PPO hyperparameters: clip range  $\epsilon = 0.2$ , GAE  $\lambda = 0.95$ , and entropy regularization.



30M-step training dynamics: upward reward progression, value convergence, and domain randomization schedule.

# Controller Architecture: Actor-Critic, Asymmetry & GRU Rationale

## Step 1: Decoupled Control

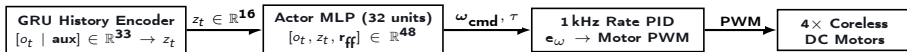
- Pure PPO mapping directly to motor PWM caused severe motor chatter and crashes.
- **Decoupled Solution:** Neural policy commands 100 Hz angular rates  $\omega_{\text{cmd}}$  and collective thrust  $\tau$ . Native 1 kHz PID stabilizes motors.

## Step 2: Asymmetric Actor-Critic

- **Critic (Trainer):** In simulation, observes 96 privileged states (motor delay, true drag, battery sag) to reduce policy gradient variance.
- **Actor (Pilot):** Receives only deployable noisy sensor readings. Privileged guidance yields robust real-world generalization.

## Step 3: Why a GRU Encoder?

- **RNN vs. LSTM:** Vanilla RNNs cannot capture long-term history due to vanishing gradients, while standard LSTMs have too many gates and cannot run effectively on the STM32 MCU.
- **GRU Hidden State:** Holds a compact 16-D hidden state  $h_t$  that captures long-term temporal dependencies recursively in-place ( $O(1)$  memory).
- **RMA Inspiration:** Provides online adaptation to mass shifts and battery sag without retraining.



# Embedded Deployment: Deterministic Execution on FreeRTOS

## Two-Rate FreeRTOS Execution Hierarchy

### ■ 100 Hz High-Level Policy Task (Every 10 ms):

- Acquires Lighthouse v2 tracking pose and BMI088 IMU readings.
- Passes data through the Estimator Plausibility Filter.
- Updates GRU hidden state vector and computes Actor MLP forward pass.
- Dispatches body rate setpoints  $\omega_{\text{cmd}}$  and collective thrust  $\tau$ .

### ■ 1 kHz Low-Level Stabilizer Task (Every 1 ms):

- Compares rate setpoints against raw 1 kHz gyro readings.
- Executes native PID feedback to generate direct motor PWM signals.

## Bare-Metal Embedded Engineering

- **Strict Static Memory Allocation:** Neural network weights and activation buffers are compiled as static C arrays ('policy\_weights.h', 'policy\_net.c').
- **Privileged Critic Discarded:** The large 96-D simulation critic is discarded post-training; only the compact actor MLP and GRU are compiled onto the microcontroller.
- **Hardware FPU Acceleration:** All inference computations evaluate single-precision floating-point operations directly on the ARM Cortex-M4 hardware FPU.

# Sim-to-Real Transfer: Overcoming the Physical Reality Gap

## The 3 Physical Roadblocks in the Lab

- **1. Actuator Phase Lag:** Original 16 mm coreless motors exhibit physical response delay ( $\tau \approx 25$  ms). At angular rates of 15–20 rad/s, this creates a **25°–30° phase lag**, triggering severe over-rotation and crashes.
- **2. Severe Battery Voltage Sag:** Demanding 100% climb throttle causes the 1S LiPo battery to collapse from 4.1 V down to  $\sim 3.1$  V (or 2.8 V in back-to-back flights), reducing available thrust by  $\sim 27\%$ .
- **3. Optical Tracking Occlusion:** During inversion, top-mounted Lighthouse sensors lose line of sight to base stations, causing standard EKF estimators to diverge.

## Key Physics Injected into Simulation

- **First-Order Actuator Lag Dynamics:**  
 $\dot{\omega}_{M,i} = \frac{1}{\tau}(\omega_{M,i}^{\text{cmd}} - \omega_{M,i})$  with randomized time constant  $\tau \in [18, 35]$  ms.
- **Coupled Battery Voltage Sag Modeling:** Modeled dynamic voltage sag as a function of instantaneous throttle draw and battery state-of-charge.
- **Aerodynamic Drag & Precession:** Coupled non-linear drag tensors and rotor gyroscopic torques during high-rate pitch maneuvers.
- **Domain Randomization:** Perturbed vehicle mass, inertia tensors, motor thrust coefficients, and sensor noise variances.

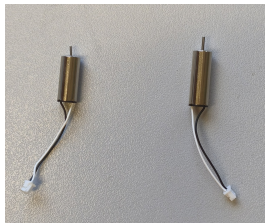
# Firmware Safeguards & Motor Transfer Adaptation

## Firmware Precautions & Safeguards

- **Crash Precautions:** Took practical firmware precautions to protect the drone and prevent occasional crashes during aggressive maneuvers.
- **Estimator Plausibility Filter:** Pre-processing filter after the Kalman estimator that rejects unphysical jumps (e.g. teleporting states). When optical tracking drops during inversion, it freezes estimates at the last valid reading and temporarily suspends false altitude aborts.
- **Debounced Ground Failsafe:** Smoothly ramps thrust to zero over 1.2s in self-leveling attitude mode instead of cutting props abruptly.
- **Iterative Lab Loop:** Flying in the safety cage, analyzing 100 Hz telemetry logs, and updating simulation parameters.

## Motor Transfer Adaptation (RMA)

- **Plant Upgrade:** Installed upgraded 20 mm motors (right) vs. 16 mm baseline (left), raising peak thrust to 1.19 N (mass: 41 g, T/W: 2.97).
- **Motor Transfer Adaptation:** Completed flips **without retraining!** The GRU latent vector  $z_t$  absorbed mass and thrust changes online.

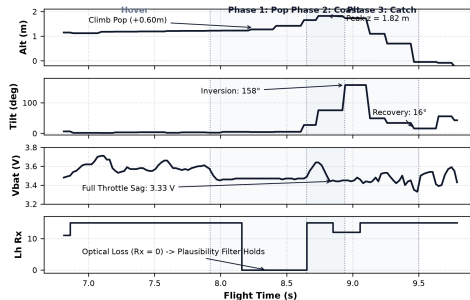


Motor upgrade: 16 mm (left) vs. 20 mm (right)

# Experimental Flight Results: Hardware Flight Log Telemetry

## Measured Hardware Maneuver Phases (Flight #6)

- **Phase 1: Climb Pop** ( $t = 7.92 - 8.65$  s) Policy commands saturated thrust; vehicle climbs from 1.22 m to peak  $z = 1.82$  m (+0.60 m pop clearance). Battery voltage sags to 3.33 V.
- **Phase 2: Inverted Ballistic Coast** ( $t = 8.65 - 8.94$  s) Thrust reduced; rapid pitch rotation reaches  $158^\circ$  inversion. Optical reception collapses to 0 packets; the Plausibility Gate holds state continuity.
- **Phase 3: Catch & Upright Recovery** ( $t = 8.94 - 9.46$  s) Counter-torque halts rotation, thrust ramps to arrest descent, recovering tilt to  $16^\circ$  and touching down upright without tumbling ('sup\_bits' remained 542).



Real 100 Hz hardware flight log (Flight #6): climb pop ( $z = 1.82$  m),  $158^\circ$  inversion tilt, battery voltage sag, and optical occlusion resilience.

# Empirical Flight Demonstration: Simulation vs. Real Flip

## MuJoCo MJX Physics Simulation



Continuous pitch flips in vectorized MJX physics environment

## Physical Hardware Flight



Autonomous onboard 360° power flip & upright catch

Click on either video to play inline in SlidePilot. (In Apple Preview, [click here to open in QuickTime](#)).

# Looking Ahead: Technical Improvements & Practical Use Cases

## Future Technical Extensions

- **Onboard Visual-Inertial Odometry (VIO):** Integrating ultra-light optical flow camera sensors to eliminate dependency on external Lighthouse tracking stations entirely.
- **Arbitrary Initial Condition Tumble Recovery:** Expanding the training curriculum to initialize episodes from arbitrary tumbling states, enabling recovery from mid-air collisions.
- **Scaling Training Compute:** Leveraging the observed upward compute-bound training dynamics by scaling training steps to further refine trajectory tracking accuracy.

## Realistic Real-World Applications

- **Agile Micro-Aerial Robotics:** Enabling autonomous micro quadcopters to navigate highly dynamic, cluttered environments with extreme agility.
- **Confined Industrial Inspection:** Navigating narrow conduits, complex pipes, and industrial machinery where physical access is strictly limited.
- **Extreme Maneuver Autonomy:** Establishing robust baseline neural controllers for aggressive aerobatic trajectories in restricted indoor flight envelopes.

# Project Summary: Autonomous Edge AI Control

## Key Project Takeaways

- **Autonomous Onboard Aerobatics:** Proved that deep reinforcement learning can control extreme  $360^\circ$  flips entirely onboard an ARM Cortex-M4 microcontroller without offboard assistance.
- **JAX Environment Parallelization:** Parallelized training rollouts, delivering a  $3\times$  to  $5\times$  speedup even on CPU.
- **RMA-Inspired Latent Adaptation:** Gated Recurrent Unit (GRU) provides compact, low-overhead online adaptation, absorbing battery sag and enabling motor transfer adaptation.
- **Firmware Safeguards:** Practical precautions that protect the drone and prevent occasional crashes during sensor dropouts.
- **Empirical Lab Validation:** Confirmed through real hardware flight logs.

### Autonomous Agile Aerobatics

- $360^\circ$  Autonomous Power Flips
- Real Micro-Quadcopter Flight Tests
  - Fully Onboard ARM Cortex-M4
  - Asymmetric Actor-Critic + GRU
  - Motor Transfer Adaptation (RMA)
- Firmware Safeguards & Plausibility Gate

### Andreas Panagopoulos

Department of ECE, University of Patras

[andreas.panagopoulos@ac.upatras.gr](mailto:andreas.panagopoulos@ac.upatras.gr)

Open-Source: [github.com/andrupe/Quadcopter-flip](https://github.com/andrupe/Quadcopter-flip)